

DockWatch

Docker Container Health Monitoring System

Software Requirements Specification

Version 1.0 | April 15, 2026

Prepared For: Software Developers, QA Engineers
System Administrators, Academic Evaluators

Prepared By: Systems Engineering Team

Reviewed By: QA & Security Leads

Status: Baseline v1.0

Next Review: Q3 2026

Contents

1	Introduction	2
1.1	Purpose	2
1.2	Scope	2
1.3	Definitions, Acronyms, and Abbreviations	2
1.4	References	3
2	Overall Description	3
2.1	Product Perspective	3
2.2	User Characteristics	3
2.3	Operating Environment & Constraints	4
2.4	Assumptions & Dependencies	4
3	Specific Requirements	4
3.1	Functional Requirements	4
3.2	Non-Functional Requirements	5
4	Appendices	5
4.1	Appendix A: Use Case Summary	5
4.2	Appendix B: Requirements Traceability Matrix	5

1 Introduction

1.1 Purpose

The purpose of this document is to define the functional and non-functional requirements for **DockWatch**, a centralized Docker container health monitoring system. DockWatch is designed to:

- Reduce system downtime through automated failure detection and recovery mechanisms.
- Provide historical performance data to support infrastructure capacity planning.
- Centralize container monitoring into a single, unified interface.
- Enable proactive infrastructure management across development and production environments.

Key Performance Objectives:

Monitor up to **50 concurrent containers** with **<5% host CPU overhead**.

Detect container failures within **30 seconds** of occurrence.

Achieve **99.9% system uptime** for the monitoring service itself.

Deliver **sub-200ms API response times** at the 95th percentile.

1.2 Scope

DockWatch will operate as a lightweight, standalone monitoring agent/service that interfaces directly with the Docker Engine. It will:

- Collect real-time telemetry and expose REST and WebSocket APIs.
- Provide a web-based UI/UX dashboard.
- Support configuration via YAML and data serialization using JSON.
- Communicate securely via TLS/JWT authentication.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
API	Application Programming Interface
CI/CD	Continuous Integration/Continuous Deployment
CPU	Central Processing Unit
Docker	Platform for developing, shipping, and running containerized applications

Term	Definition
JWT	JSON Web Token
REST	Representational State Transfer
SRS	Software Requirements Specification
TLS	Transport Layer Security
WebSocket	Protocol providing full-duplex communication over TCP
YAML	YAML Ain't Markup Language

1.4 References

1. IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*
2. Docker Engine API v1.41 Documentation
3. OWASP Top 10 Security Risks
4. ISO/IEC 25010:2011, *Systems and Software Quality Requirements and Evaluation*

2 Overall Description

2.1 Product Perspective

DockWatch operates as a complementary monitoring layer within existing Docker-based infrastructure. It communicates with the Docker Engine API v1.41 to poll container states, resource utilization (CPU, RAM, I/O), and health check statuses. The system exposes a RESTful API for external integrations and a WebSocket endpoint for real-time dashboard updates.

2.2 User Characteristics

User Type	Description
Software Developers	Utilize historical performance data and API access to optimize container configurations.
QA Engineers	Leverage monitoring logs and failure timelines to reproduce edge cases.
System Administrators	Primary operators who configure alerts and monitor real-time infrastructure health.

User Type	Description
Academic Evaluators	Reference system architecture and performance metrics for research purposes.

2.3 Operating Environment & Constraints

- **Runtime:** Linux-based hosts running Docker Engine (API v1.41+)
- **Network:** Requires HTTP/HTTPS and WebSocket connectivity secured via TLS.
- **Security:** Authentication via JWT; adherence to OWASP Top 10 mitigation practices.
- **Resource Constraints:** Must maintain <5% CPU overhead monitoring up to 50 containers.
- **Reliability:** Monitoring service must achieve 99.9% uptime.

2.4 Assumptions & Dependencies

- Docker daemon is running and API socket is accessible.
- Host system maintains stable network connectivity.
- Configuration files are provided in valid YAML format.

3 Specific Requirements

3.1 Functional Requirements

ID	Requirement	Priority
FR-01	System shall continuously poll Docker Engine API to discover running containers and track PID, CPU, and RAM usage.	High
FR-02	System shall detect container failures within 30 seconds.	High
FR-03	System shall automatically trigger predefined recovery actions (restart, isolate, or notify) based on configurable rules.	High
FR-04	System shall store historical performance metrics and expose data via REST API.	Medium
FR-05	System shall provide a centralized UI/UX dashboard displaying real-time status, alerts, and historical trends.	High

ID	Requirement	Priority
FR-06	System shall support configuration via YAML files and export telemetry in JSON format.	Medium

3.2 Non-Functional Requirements

ID	Category	Requirement	Target
NFR-01	Performance	API response time	≤200ms at 95th percentile
NFR-02	Resource	Host CPU overhead	<5% for 50 containers
NFR-03	Availability	Monitoring service uptime	99.9%
NFR-04	Security	Authentication & Transport	JWT + TLS 1.2+
NFR-05	Scalability	Concurrent container monitoring	Up to 50 per agent
NFR-06	Maintainability	Compliance & Documentation	ISO/IEC 25010, IEEE 830
NFR-07	Interoperability	API & Data Formats	REST, JSON, YAML, Web-Socket

4 Appendices

4.1 Appendix A: Use Case Summary

UC-01. Admin configures monitoring thresholds via YAML → System validates and applies rules → Dashboard reflects new policies.

UC-02. Container crashes → DockWatch detects failure within 30s → Triggers restart & logs event → Sends alert via API/WebSocket.

UC-03. QA engineer queries historical CPU/RAM data via REST API → Receives JSON payload → Uses data for performance regression testing.

4.2 Appendix B: Requirements Traceability Matrix

Requirement	Stakeholder	Design Phase	Test Method
FR-01, NFR-02	Sys Admins, Devs	Architecture	Load Testing
FR-02, FR-03	QA, Sys Admins	Failure Simulation	Chaos Engineering
FR-04, FR-05	Evaluators, QA	Data Layer	API Validation
NFR-03, NFR-04	All	Security & HA	Penetration & Uptime